Trabajo Práctico: Procedimientos Almacenados Avanzados

Bases de Datos II | UTN Mar del Plata

Esquema de Base de Datos

```
CREATE TABLE socios (
    id_socio INT PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(100),
    apellido VARCHAR(100),
    fecha_nacimiento DATE,
    direccion VARCHAR(200),
    telefono VARCHAR(20)
);

CREATE TABLE planes (
    id_plan INT PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(100),
    precio DECIMAL(10,2)
);

CREATE TABLE actividades (
    id_actividad INT PRIMARY KEY AUTO_INCREMENT,
    id_socio INT,
    id_plan INT,
    fecha DATE,
    descripcion VARCHAR(200),
    FOREIGN KEY (id_socio) REFERENCES socios(id_socio) ON DELETE CASCADE,
    FOREIGN KEY (id_plan) REFERENCES planes(id_plan)
);
```

Parte 1 — Manejo de Errores con Handlers

Ejercicio 1: insertarEnTablaInexistente

Enunciado: Crear un procedimiento almacenado que intente insertar un registro en una tabla inexistente y maneje el error con un handler SQLEXCEPTION.

```
DELIMITER //

CREATE PROCEDURE insertarEnTablaInexistente()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        SELECT 'Se ha producido un error SQL' AS Mensaje;
    END;

    INSERT INTO tabla_inexistente (columna) VALUES ('valor');

    SELECT 'Inserción completada' AS Mensaje;
END //

DELIMITER ;
```

Ejercicio 2: crearTablaExistente

Enunciado: Crear un procedimiento que intente crear una tabla que ya existe y maneje la advertencia con un handler SQLWARNING.

```
DELIMITER //

CREATE PROCEDURE crearTablaExistente()
BEGIN
    DECLARE CONTINUE HANDLER FOR SQLWARNING
    BEGIN
        SELECT 'Advertencia: La tabla ya existe' AS Mensaje;
    END;

    CREATE TABLE ejemplo_table (
        id INT AUTO_INCREMENT PRIMARY KEY,
        nombre VARCHAR(50)
    );

    SELECT 'Creación de tabla completada' AS Mensaje;
END //

DELIMITER ;
```

Ejercicio 3: seleccionarRegistroInexistente

Enunciado: Crear un procedimiento que intente seleccionar un registro de una tabla basándose en un ID que no existe y maneje la condición NOT FOUND.

```
DELIMITER //

CREATE PROCEDURE seleccionarRegistroInexistente()
BEGIN
    DECLARE v_nombre VARCHAR(100);
    DECLARE CONTINUE HANDLER FOR NOT FOUND
    BEGIN
        SELECT 'No se encontraron registros con el ID especificado' AS Mensaje;
    END;

    SELECT nombre INTO v_nombre FROM ejemplo_table WHERE id = 9999;

    SELECT 'Selección completada' AS Mensaje;
END //

DELIMITER ;
```

Ejercicio 4: manejoCombinado

Enunciado: Crear un procedimiento que intente insertar en una tabla inexistente y luego intente crear una tabla que ya existe, manejando ambos casos con sus respectivos handlers.

```
DELIMITER //

CREATE PROCEDURE manejoCombinado()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        SELECT 'Se ha producido un error SQL' AS Mensaje;
    END;

    DECLARE CONTINUE HANDLER FOR SQLWARNING
    BEGIN
        SELECT 'Advertencia: La tabla ya existe' AS Mensaje;
    END;

    INSERT INTO tabla_inexistente (columna) VALUES ('valor');

    CREATE TABLE ejemplo_table (
        id INT AUTO_INCREMENT PRIMARY KEY,
        nombre VARCHAR(50)
    );

    SELECT 'Operaciones completadas' AS Mensaje;
END //

DELIMITER ;
```

Parte 2 — Procedimientos con el esquema del gimnasio

Ejercicio 5: insertarActividad (FK inexistente)

Enunciado: Crear un procedimiento almacenado que reciba `id_socio` y `id_plan` e intente insertar un registro en la tabla `actividades`. Debe manejar el error si alguno de los IDs no existe en sus respectivas tablas.

```
DELIMITER //

CREATE PROCEDURE insertarActividad(
    IN p_id_socio INT,
    IN p_id_plan INT
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        SELECT 'Error: El socio o el plan no existen.' AS Mensaje;
    END;

    INSERT INTO actividades (id_socio, id_plan, fecha, descripcion)
    VALUES (p_id_socio, p_id_plan, CURDATE(), 'Actividad generica');
END //

DELIMITER ;
```

Ejercicio 6: seleccionarSocio (NOT FOUND)

Enunciado: Crear un procedimiento que reciba un ID de socio e intente seleccionar sus datos. Si el socio no existe, debe manejar la condición NOT FOUND.

```
DELIMITER //

CREATE PROCEDURE seleccionarSocio(IN p_id_socio INT)
BEGIN
    DECLARE v_nombre VARCHAR(100);
    DECLARE v_apellido VARCHAR(100);

    DECLARE CONTINUE HANDLER FOR NOT FOUND
    BEGIN
        SELECT 'No se encontró un socio con el ID especificado.' AS Mensaje;
    END;

    SELECT nombre, apellido INTO v_nombre, v_apellido
    FROM socios
    WHERE id_socio = p_id_socio;

    SELECT v_nombre AS Nombre, v_apellido AS Apellido;
END //

DELIMITER ;
```

Parte 3 — Manejo de Transacciones

Ejercicio 7: registrarSocioConPlan

Enunciado: Desarrollar un procedimiento almacenado que reciba los datos de un nuevo socio (nombre, apellido, fecha_nacimiento, dirección, teléfono), el ID de un plan y una descripción de la actividad. Debe insertar el socio, luego insertar una actividad asociada al nuevo socio y al plan indicado, con la descripción proporcionada. Si ocurre un error, debe realizar ROLLBACK y mostrar mensaje de error. Si todo es exitoso, COMMIT y mensaje de éxito.

```
DELIMITER //

CREATE PROCEDURE registrarSocioConPlan(
    IN p_nombre VARCHAR(100),
    IN p_apellido VARCHAR(100),
    IN p_fecha_nacimiento DATE,
    IN p_direccion VARCHAR(200),
    IN p_telefono VARCHAR(20),
    IN p_id_plan INT,
    IN p_descripcion_actividad VARCHAR(200)
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SELECT 'Error: No se pudo registrar el socio o la actividad. Operación cancelada.' AS Mensaje;
    END;

    START TRANSACTION;

    INSERT INTO socios (nombre, apellido, fecha_nacimiento, direccion, telefono)
    VALUES (p_nombre, p_apellido, p_fecha_nacimiento, p_direccion, p_telefono);

    INSERT INTO actividades (id_socio, id_plan, fecha, descripcion)
    VALUES (LAST_INSERT_ID(), p_id_plan, CURDATE(), p_descripcion_actividad);

    COMMIT;
    SELECT 'Socio y actividad registrados con éxito.' AS Mensaje;
END //

DELIMITER ;
```

Ejercicio 8: actualizarPlanYRegistrarActividad

Enunciado: Desarrollar un procedimiento que reciba el ID de un plan, su nuevo precio, el ID de un socio y una descripción de la actividad. Debe actualizar el precio del plan e insertar una nueva actividad para ese socio y plan, con la descripción indicada. Manejar errores con ROLLBACK. Si todo es correcto, COMMIT y mensaje de éxito.

```
DELIMITER //

CREATE PROCEDURE actualizarPlanYRegistrarActividad(
    IN p_id_plan INT,
    IN p_nuevo_precio DECIMAL(10,2),
    IN p_id_socio INT,
    IN p_descripcion_actividad VARCHAR(200)
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SELECT 'Error: No se pudo actualizar el plan o registrar la actividad. Operación cancelada.' AS Mensaje;
    END;

    START TRANSACTION;

    UPDATE planes SET precio = p_nuevo_precio WHERE id_plan = p_id_plan;

    INSERT INTO actividades (id_socio, id_plan, fecha, descripcion)
    VALUES (p_id_socio, p_id_plan, CURDATE(), p_descripcion_actividad);

    COMMIT;
    SELECT 'Plan actualizado y actividad registrada con éxito.' AS Mensaje;
END //

DELIMITER ;
```

Ejercicio 9: eliminarSocioYActividades

Enunciado: Desarrollar un procedimiento que reciba el ID de un socio. Debe eliminar todas las actividades asociadas a ese socio y luego eliminar el socio. Si el socio no existe, debe realizar ROLLBACK y mostrar mensaje de error. En caso de cualquier error en las eliminaciones, también debe hacer ROLLBACK.

```
DELIMITER //

CREATE PROCEDURE eliminarSocioYActividades(IN p_id_socio INT)
BEGIN
    DECLARE v_existe INT DEFAULT 0;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SELECT 'Error durante la eliminación. Transacción revertida.' AS Mensaje;
    END;

    START TRANSACTION;

    SELECT COUNT(*) INTO v_existe FROM socios WHERE id_socio = p_id_socio;

    IF v_existe = 0 THEN
        ROLLBACK;
        SELECT 'Error: El socio no existe. Operación cancelada.' AS Mensaje;
    ELSE
        DELETE FROM actividades WHERE id_socio = p_id_socio;
        DELETE FROM socios WHERE id_socio = p_id_socio;

        COMMIT;
        SELECT 'Socio y sus actividades eliminados con éxito.' AS Mensaje;
    END IF;
END //

DELIMITER ;
```